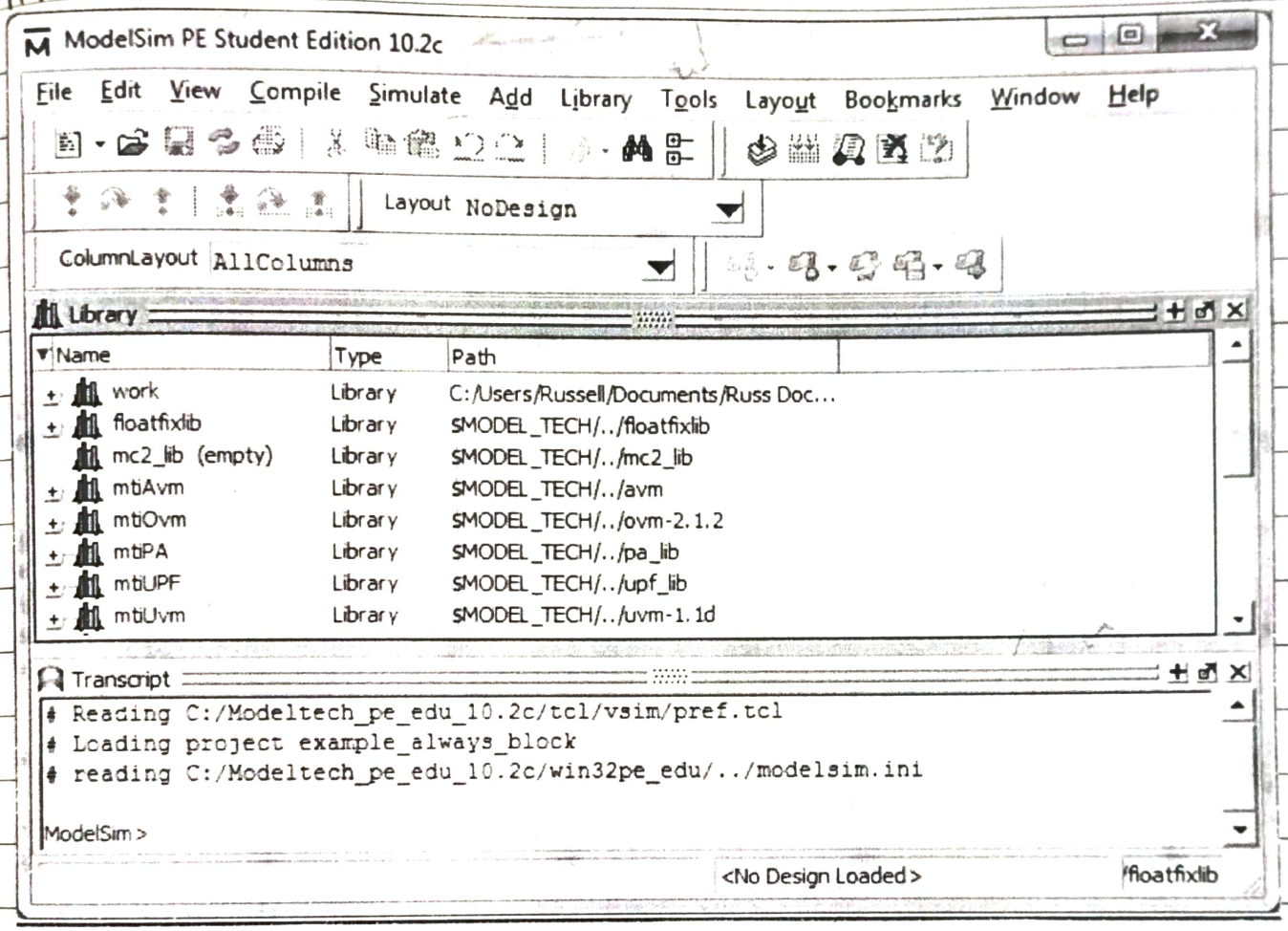
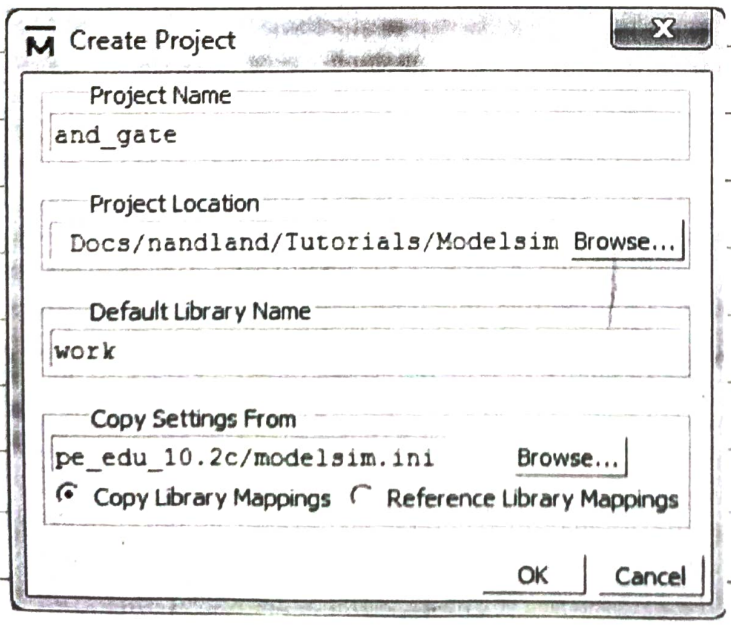
EXPERIMENT - 11

Introduction to Modelsim

while opening Modelsim you will see a window like this.

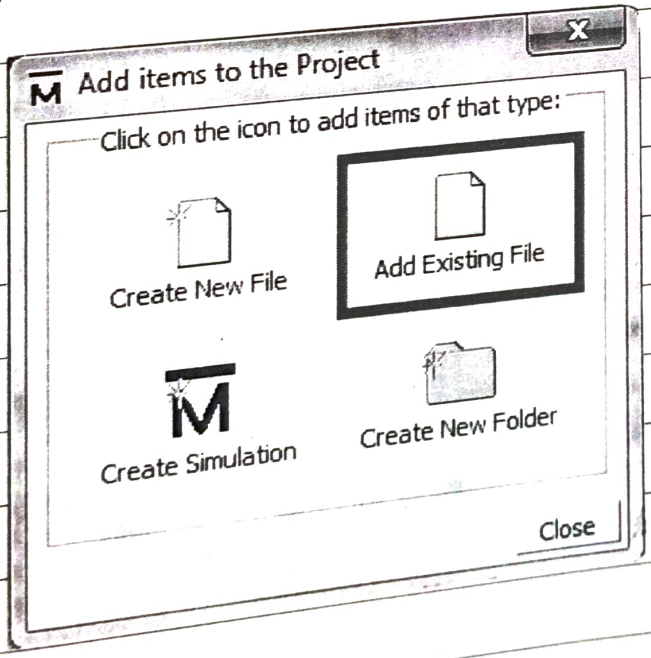


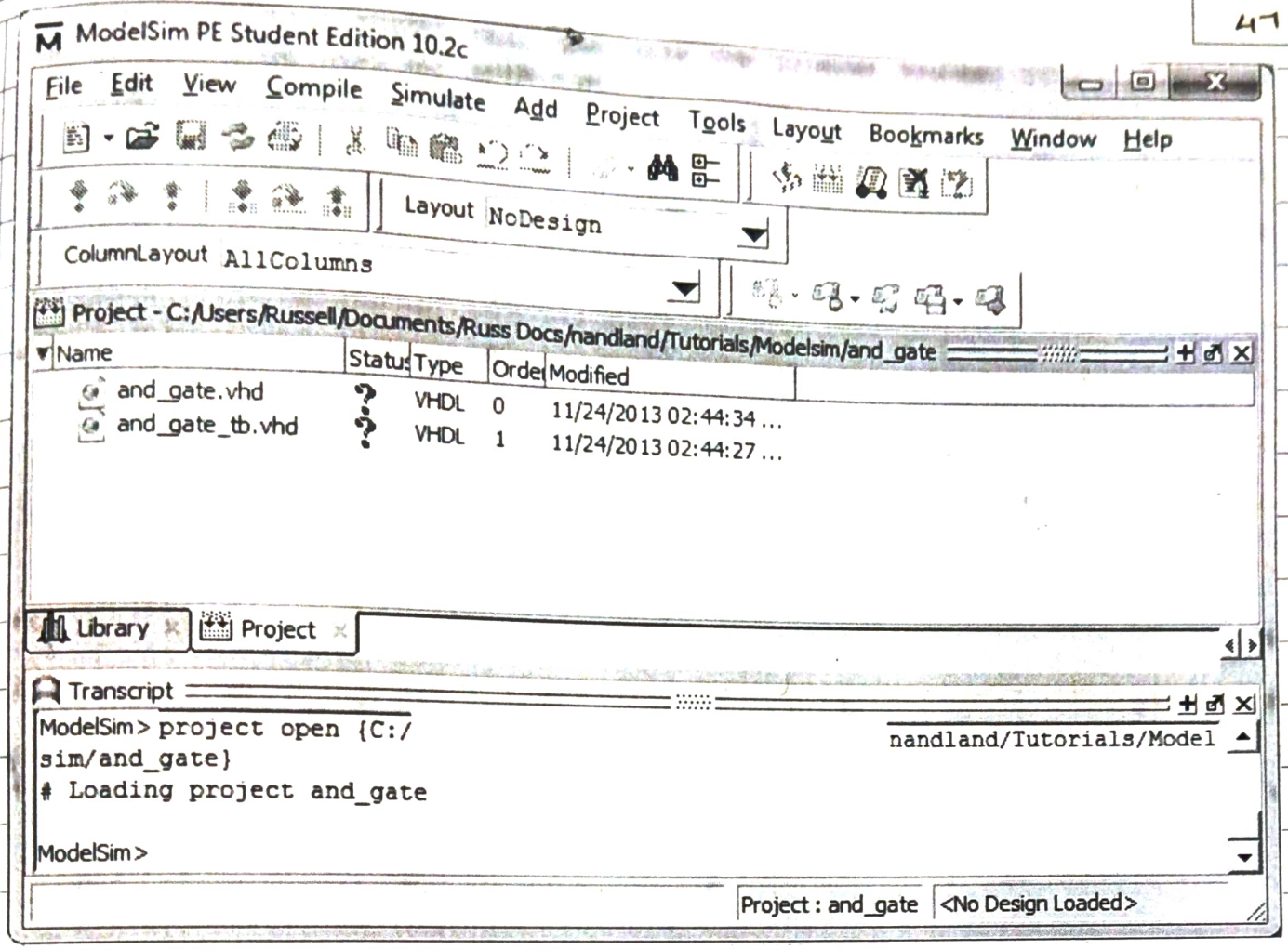
Modelsim Main Window



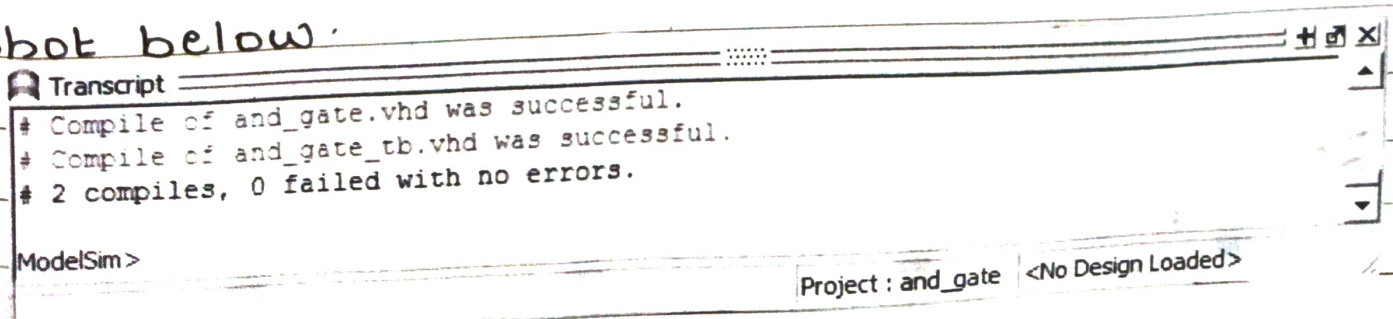
In order to run your simulation, you need to create a project. Click File → New → Project. You will see the window presented on the left. Choose a location for your new project and give it the name and-gate. Projects in Modelsim have the file extension .prj. Leave the other settings to their default. This just says that all code will be compiled into the library "work".

Click on create New File as shown in the picture to the right. Click OK when done.

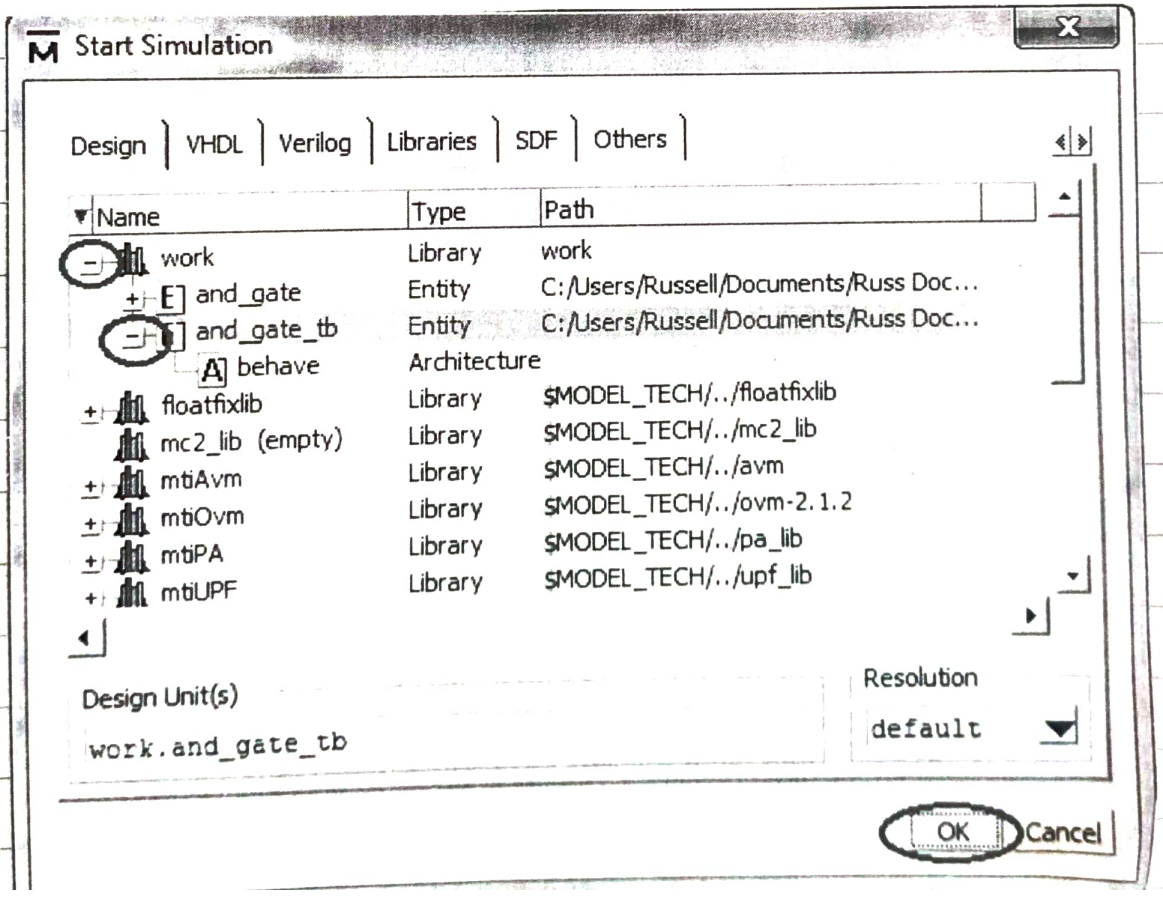


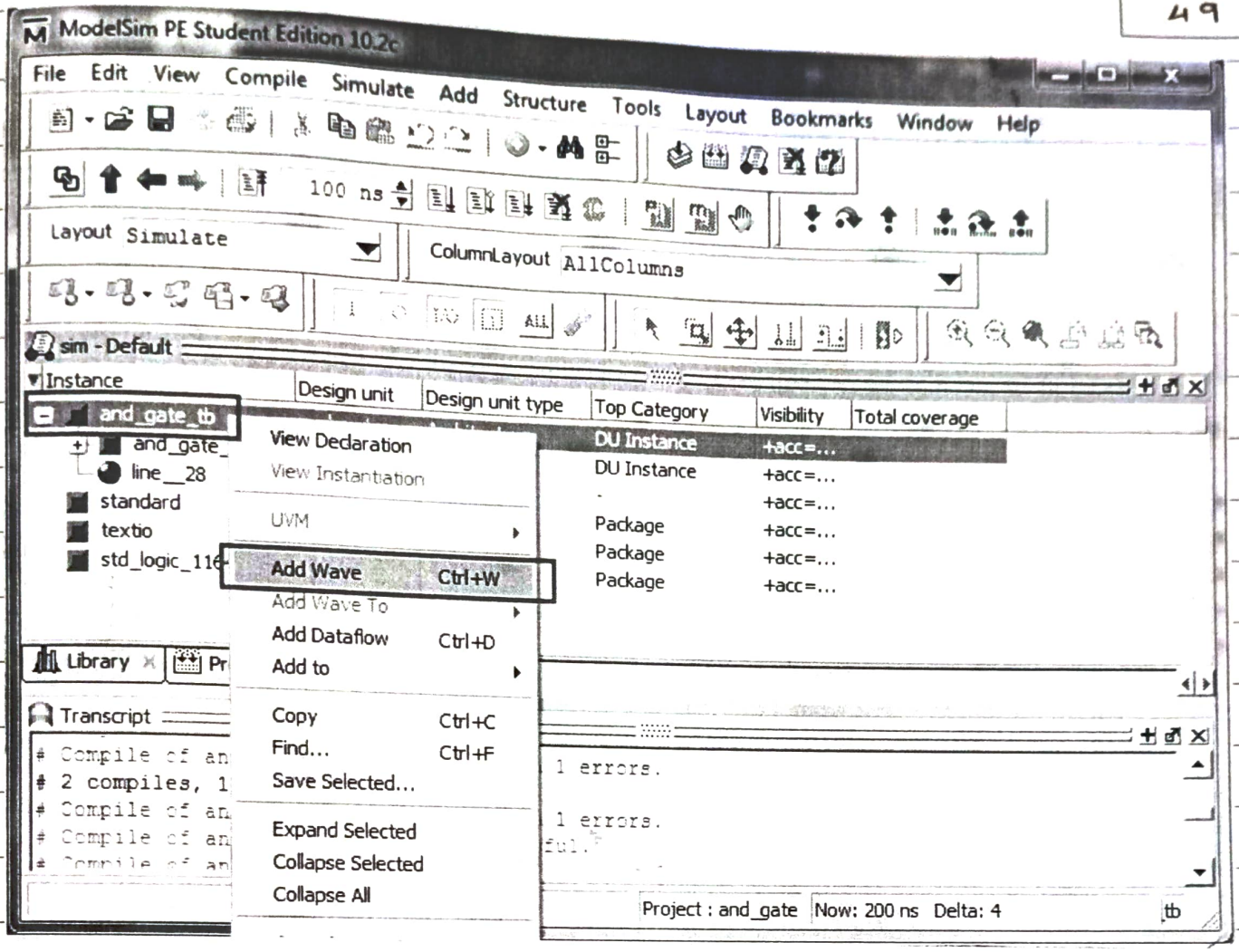


Notice now that the files have been added successfully to your project. See those two blue question marks in the Modelsim Project window Figure above? That means that Modelsim has not compiled the files yet. You will need to compile the source files. To do this, right click on & and_gate.vhd, click on compile, then click on compile All. You should see messages in the console window appear in green that the compile was successful as shown in the screen-shot below.



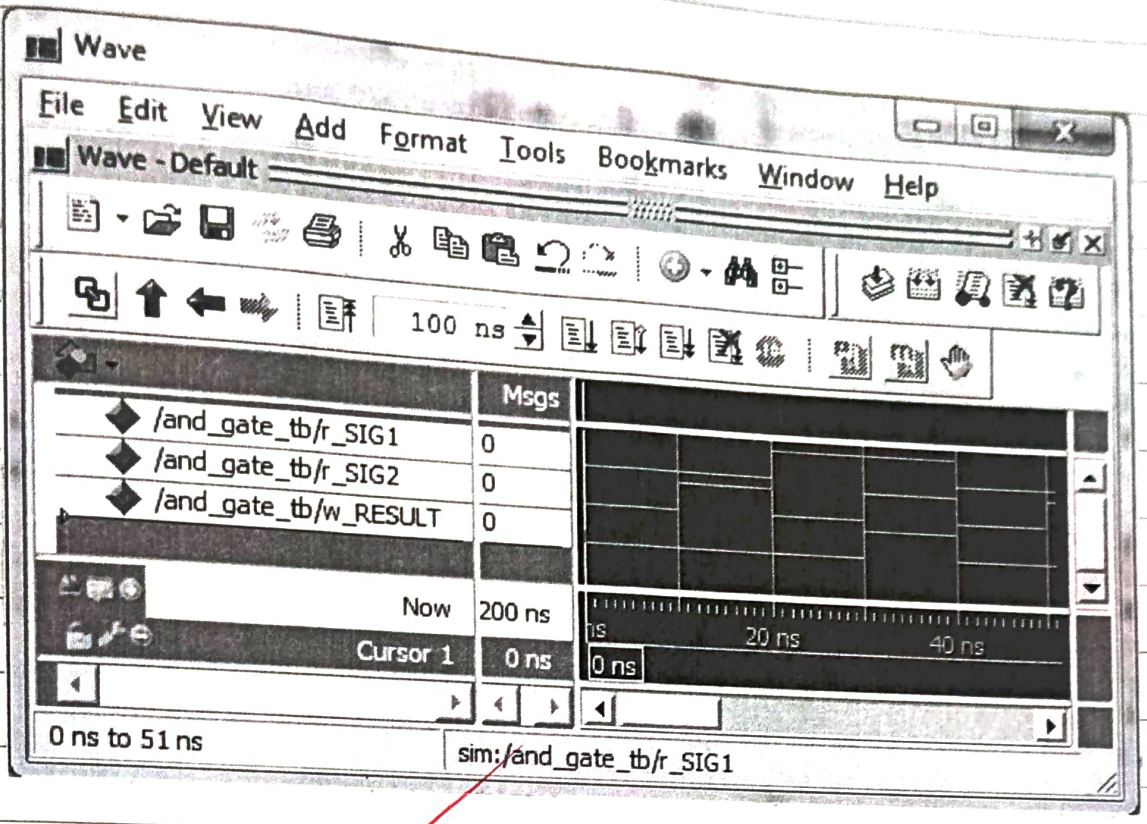
To start your simulation, click on Simulate in the Menu Bar, then click Start Simulation. This opens the Start Simulation window. Click on the plus sign next to work, then click on the plus sign next to and-gate-tb. Make sure you select and-gate-tb and not and-gate as we want to simulate the design at the test bench level. Once and-gate-tb is highligh ted, click OK.





The simulation is ready and waiting. Now, the majority of the time that you use Modelsim will be spent looking at the waveform view. The waveform view contains waves (binary 0's and 1's, hexadecimal digits, binary digits, enumerated types, etc) for all of the signals in your design. It shows how your module reacts to different stimulus. The next figure shows you what your waveform view looks like, but first you need to add some signals to monitor. In this example, we will monitor all of the signals in the test bench. To do this, right click on and-gate-tb in the sim window and click Add wave. You can also click and drag signals to the waveform window from other

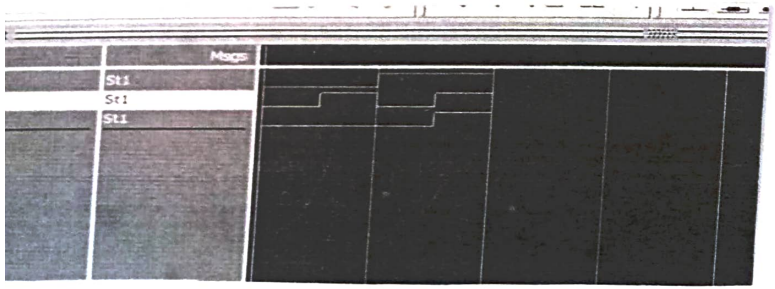
in modelsim.

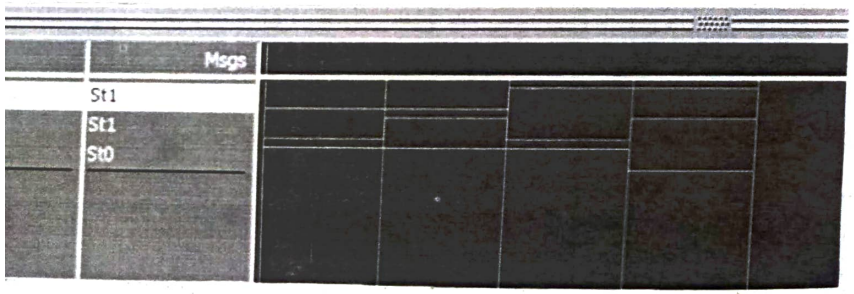


Result:

Program executed successfully.

put





EXPERIMENT-12

Development of Verilog modules for Logic Gates

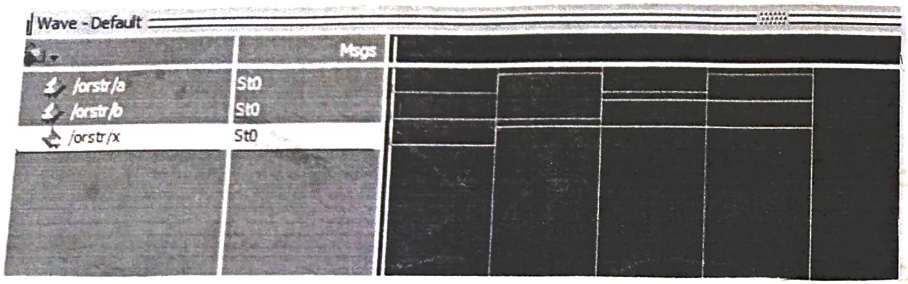
PROGRAM

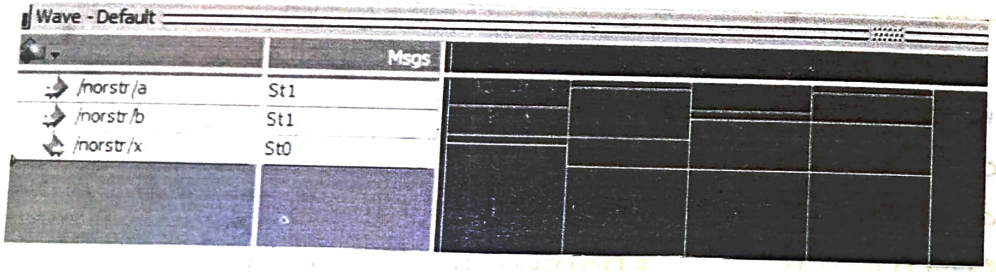
a AND Gate

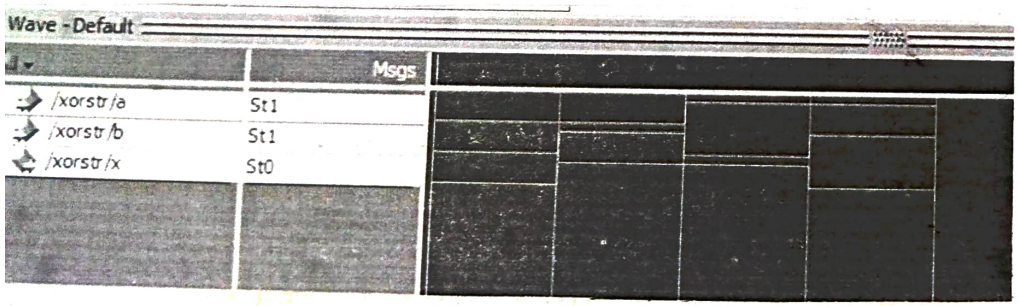
Structural Model	Data Flow model	Behavioural Model
module andstr(a, b, x);	module anddf(a, b, x);	module andbeh(a,b, x);
input a,b;	input a,b;	input a,b;
output x;	Output x;	output x;
and g1(x,a,b);	assign x = (a&b);	always@(a,b)
end module	endmodule	x = a&b;
		endmodule

b NAND Gate

Structural model	Dataflow model	behavioural model
module nandstr(a,b,x);	module nanddf(a,b,x);	module nandbeh(a,b,x);
input a,b;	input a,b;	input a,b;
output x;	output x;	output x;
nand g1(x,a,b);	assign x = !(a&b);	reg x;
endmodule	endmodule	always @(a,b)
		x = !(a&b);
		endmodule







c. OR Gate

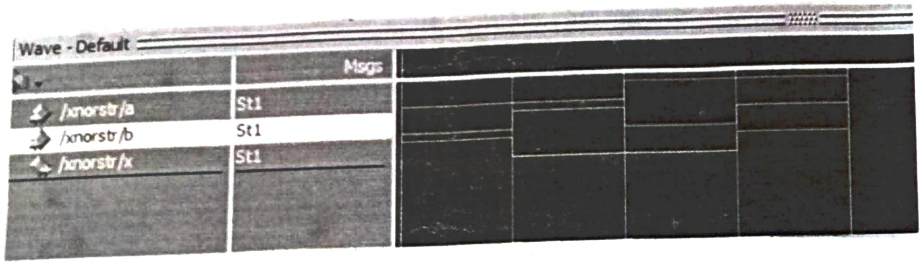
Structural model	Data flow model	Behavioural model
module orstr(a, b, x);	module ordf(a,b, x);	module orbeh(a,b,x);
input a,b;	input a,b;	input a,b;
output x;	output x;	output x;
or gl(x,a,b);	assign x=(a|b);	reg x;
endmodule	endmodule	always@(a,b)
		x = a|b;
		endmodule

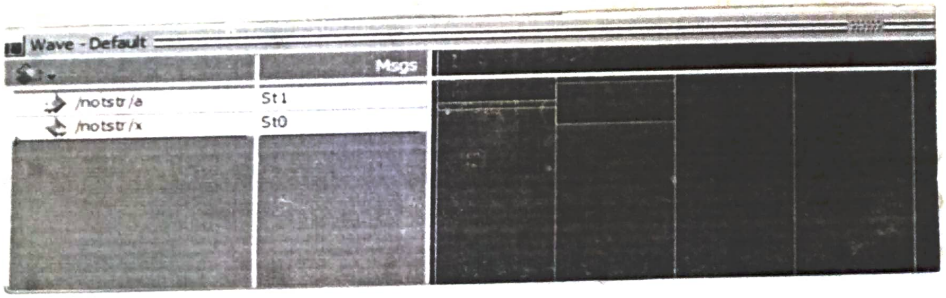
d. NOR Gate

Structural model	Dataflow model	Behavioural model
module norstr(a,b,x);	module nordf(a,b,x);	module norbeh(a,b,x);
input a,b;	input a,b;	input a,b;
output x;	output x;	output x;
nor gl(x,a,b);	assign x=!(a|b);	reg x;
endmodule	endmodule	always@(a,b);
		x = !(a|b);
		endmodule

e. XOR Gate

Structural model	Dataflow model	Behavioural model
module xorstr(a,b,x);	module xordf(a,b,x);	module xorbeh(a,b,x)
input a,b;	input a,b;	input a,b;
output x;	output x;	output x;
xor gl(x,a,b);	assign x=(a^b);	reg x;
endmodule	endmodule	always @(a,b);
		x = a^b;
		endmodule





1 XNOR Gate

Structural model	Dataflow model	Behavioural model
module xnorstr(a,b,x);	module xnordf(a,b,x);	module xnorbeh(a,b, x);
input a,b;	input a,b;	input a,b;
output x;	output x;	output x;
xnor g1(x,a,b);	assign x=!(a^b);	reg x;
endmodule	endmodule	always@(a,b)
		x=!(a^b);
		endmodule

9 NOT Gate

Structural model	Dataflow model	Behavioural model
module notstr(a,b,x);	module notdf(a,x);	module notbeh(a,x);
input a,b;	input a;	input a;
output x;	output x;	output x;
not g1 (x,a);	assign x=!a;	reg x;
endmodule	endmodule	always@(a)
		x=!a;
		endmodule

Result:

Verilog programs for all the logic gates were written and output verified.

expected output table

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

11/10/25

50

Model a Given Boolean function (sop & pos) in verilog

Aim:-

To implement a given Boolean function in Verilog HDL using different modeling styles-logical operators, Conditional operators and gate level primitives.

Apparatus required:-

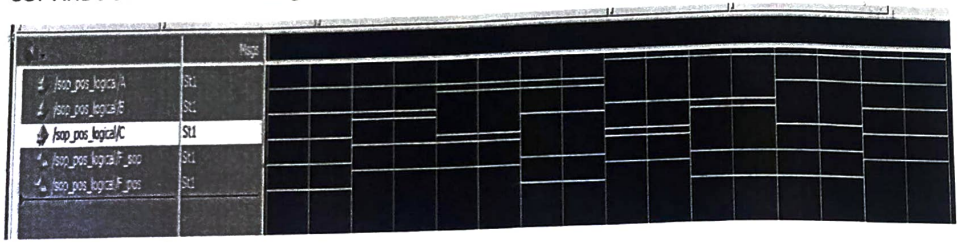
Computer with Verilog Simulation Software (Modelsim/ Xilinx ISE)

Theory:-

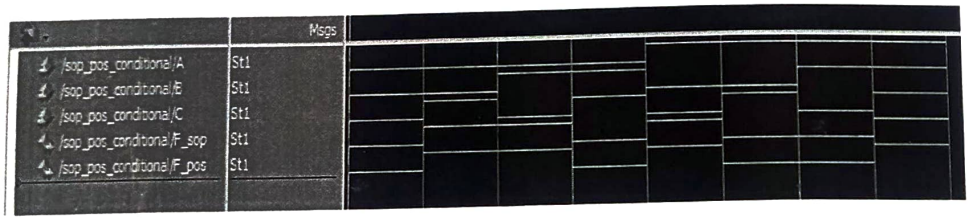
A Boolean function can be expressed in sum of products (SOP) or product of sums (POS) form. Verilog allows us to describe such functions using:

1. Continuous Assignment with logical operations:
   → Use the assign statement and operators like &, |, ~.
   eg: assign F = (A&B)|(~A&C);
2. Continuous Assignment with Conditional operators:
   → Uses the ternary operator (?:) to describe Conditional logic
   eg: assign F = (A&B)?1:0;
3. Gate_level primitives:
   → Implements the Boolean function using built-in gate Primitives like and, or, not etc
   eg:- and (w, A, B);

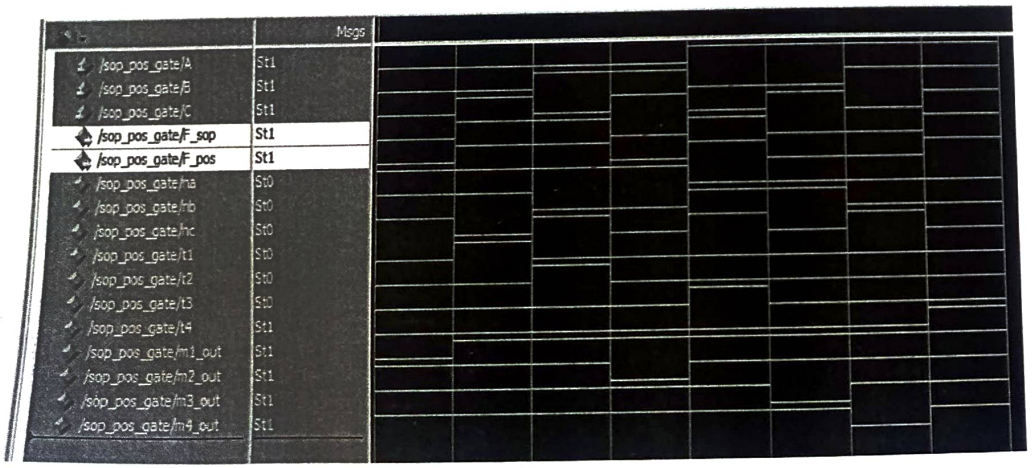
SOP AND POS Continuous Assignment with logical operator



SOP AND POS Continuous Assignment with Conditional Operators:



SOP AND POS Gate-level Primitives:



or(F,w,c);

Given Boolean function:

let the given function be:

F(A,B,C) = Σ(1,2,4,7)

Boolean function (SOP and POS) in Verilog using Continuous assignment with Conditional operators F=Σm(1,2,4,7):

```
module SOP-POS-Conditional(F-Sop, Epos, A,B,C);
input A,B,C;
output F-SOP, F-POS;
// SOP: direct check of minterms (?:m)
assign F-SOP = ((~A & ~B & C)?1'b1:
                (~A & B & ~C)?1'b1:
                (A & ~B & ~C)?1'b1:
                (A & B & C)?1'b1: 1'b0);
//Pos: direct check of maxterms (?:m)
assign F-pos = ((A|B|C) & (A|~B|~C) &
                (~A|B|~C) & (~A|~B|C))?1'b1: 1'b0;
endmodule
```

Boolean function (SOP and POS) in verilog using continuous assignment with logical operators:-

// F(A,B,C) = Πm(0,3,5,6)

```
module SOP-POS-logical(F-SOP, F-POS, A,B,C);
input A,B,C;
output F-SOP, F-POS;
// SOP form: F = A'B'C + AB'C' + AB'C' + ABC
assign F-SOP = (~A & ~B & C) | (~A & B & ~C) |
(A & ~B & ~C) | (A & B & C);
```

// Pos form : F = (A+B+C)(A+B'+C')(A'+B+C')(A'+B'+C)
assign F-pos = (A|B|C) & (A|~B|~C) & (~A|B|~C) &
(~A|~B|C);
endmodule

Boolean function (SOP and POS) in Verilog using gate level Primitives.

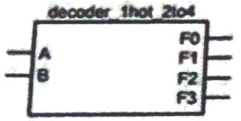
`timescale 1ns/1ps
module SOP-POS-gate (F-SOP, F-POS, A, B, C);
input A, B, C;
output F-sop, F-POS;
wire na, nb, nc;
wire t1, t2, t3, t4;
wire m1-out, m2-out, m3-out, m4-out;
// Invert-ed Signals.
not (na, A);
not (nb, B);
not (nc, C);
//SOP : F = m(0,3,5,6)
and(t1, na, nb, c);
and (t2, na, B, nc);
and (t3, A, nb, nc);
and (t4, A, B, C);
or (F-SOP, t1, t2, t3, t4);
//POS : F = M(0,3,5,6)
or (m1-out, A, B, C);
or (m2-out, A, nb, nc);
or (m3-out, na, B, nc);
or (m4-out, na, nb, C);

and (f-pos, m1-out, m2-out, m3-out, m4-out),
end module

Result:
The given boolean function was successfully modeled and verified in Verilog using
1. Continuous assignment with logical operators.
2. Continuous assignment with conditional operators.
3. Gate-level Primitives.

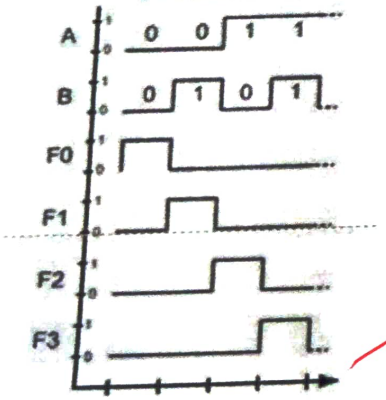
Example: 2-to-4 One-Hot Decoder - Logic Synthesis by Hand

The block diagram and truth table for this system are as follows:



A	B	F3	F2	F1	F0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Timing Waveform



Experiment: 14

11 / 20 / 25

54

Model a 4:1 Multiplexer 1:4 Demultiplexer, 4 to 2 encoder, 2 to 4 Decoder and 7-Segment Display Decoder in Verilog.

Aim:

To design and Simulate basic Combinational Circuits MUX, DEMUX, Encoder, Decoder and 7-segment display decoder using Verilog HDL with both logical and Conditional Continuous assignments.

Apparatus required:

Computer with Verilog Simulation Software (Modelsim / Xilinx ISE)

Theory:

Decoders

→ A Decoder is a digital logic Circuit with n input lines and upto 2^n output lines.

→ It Converts a binary input code into a unique active output signal, meaning only one output signal is activated at a time for each input combination.

→ A decoder essentially performs the reverse function of an encoder.

Encoders

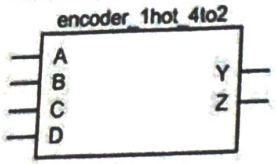
→ An encoder is a Combinational logic Circuit that Converts an active input signal into a Coded binary output.

→ It has 2^n input lines and n output lines.

→ At any given time, only one input is active (High) and the encoder generates the binary Cod

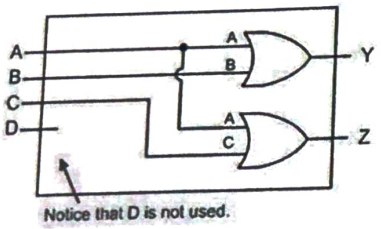
Example: 4-to-2 Binary Encoder – Logic Synthesis by Hand

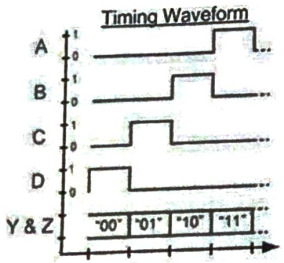
The block diagram and truth table for this system are as follows:



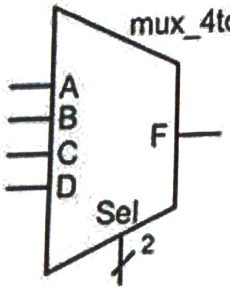
A	B	C	D	Y	Z
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

D => "00"
C => "01"
B => "10"
A => "11"



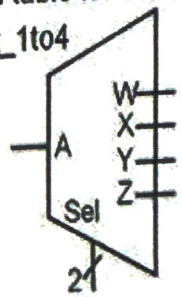


The symbol and truth table for the 4-to-1 multiplexer are as follows:



Sel	F
"00"	A
"01"	B
"10"	C
"11"	D

The symbol and truth table for the 1-to-4 demultiplexer are as follows:



Sel	W	X	Y	Z
"00"	A	0	0	0
"01"	0	A	0	0
"10"	0	0	A	0
"11"	0	0	0	A

corresponding to that input line.

Multiplexer

→ A multiplexer (mux) is a digital circuit that select one input from many and passes it to a single output.
→ It functions like a digital switch, routing only the chosen input to the output.
→ A multiplexer with n select lines can control 2^n inputs
→ It always has one output line.
→ The selection of which input to pass is determined by the binary value of the select lines.

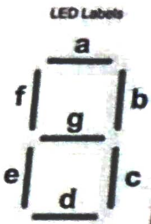
Demultiplexer

→ A Demultiplexer (DEMUX) works in the opposite (complementary) way to a multiplexer.
→ A multiplexer selects one of many inputs and forwards it to a single output.
→ A demultiplexer takes a single input and routes it to one of many outputs.

7-Segment Display Decoder

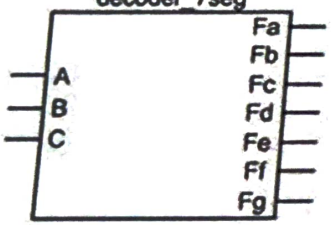
A 7-segment display decoder is a circuit used to drive character displays that are commonly found in applications such as digital clocks and household appliances. A character

A	B	C	F_a	F_b	F_c	F_d	F_e	F_f	F_g
0	0	0	1	1	1	1	1	1	0
0	0	1	0	1	1	0	0	0	0
0	1	0	1	1	0	1	1	0	1
0	1	1	1	1	1	1	0	0	1
1	0	0	0	1	1	0	0	1	1
1	0	1	1	0	1	1	0	1	1
1	1	0	1	0	1	1	1	1	1
1	1	1	1	1	1	0	0	0	0



Example: 7-Segment Display Decoder – Logic Synthesis by Hand

The block diagram and truth table for this system are as follows:



A	B	C	Fa	Fb	Fc	Fd	Fe	Ff	Fg
0	0	0	1	1	1	1	1	1	0
0	0	1	0	1	1	0	0	0	0
0	1	0	1	1	0	1	1	0	1
0	1	1	1	1	1	1	0	0	1
1	0	0	0	1	1	0	0	1	1
1	0	1	1	0	1	1	0	1	1
1	1	0	1	0	1	1	1	1	1
1	1	1	1	1	1	0	0	0	0

Each output of the decoder needs its own logic expression.

display is made up of seven individual LED's, typically labeled a–g. The input to the decoder is the binary equivalent of the decimal or hexadecimal character that is to be displayed. The output of the decoder is the arrangement of LEDs that will form the character. Decoders with 2 inputs can drive character "0" to "3". Decoder with 3 inputs can drive character "0" to "7". Decoder with 4 inputs can drive character "0" to "F" with case of hexa ch... being "A, b, C, or c, d, E and F.

Program

model a 4:1 MUX in Verilog using

a. Continuous assignment with logical operators

b. Continuous assignment with conditional operators.

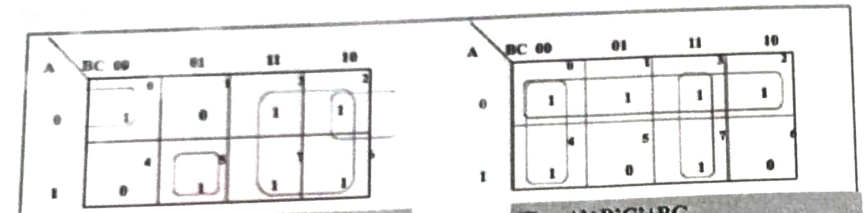
Use escale 1ns/1ps.

// 4:1 Multiplexer using logical operator

```
module mux 4to1_logical (Y,D0,D1,D2,D3,S1,S0)
  input D0,D1,D2,D3,S1,S2.
  Output Y;
assign Y=(~S1 &~S0 &D0)|(~S1&S0 &D1)|
         (S1 &~S0 &D2)|(S1&S0&D3);
endmodule.
```

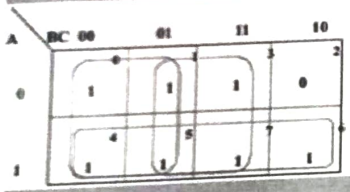
// 4:1 multiplexer using conditional operator.

```
module mux 4to1_cond (Y,D0,D1,D2,D3,S1,S0);
  Input D0,D1,D2,D3,S0,S1;
  output Y;
assign Y=(S1==0 && S0==0)?D0:
```

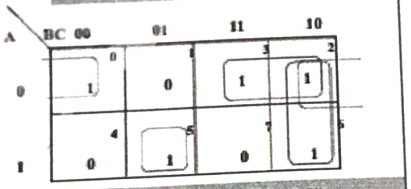


Fa = B+A'C'+AC

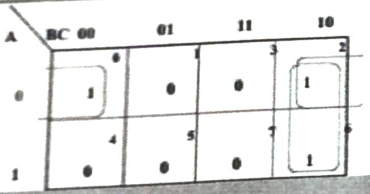
Fb = A'+B'C'+BC



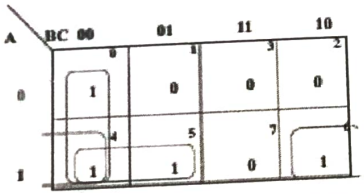
Fc = B'+C+A



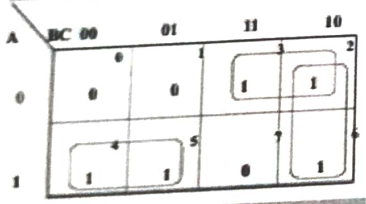
Fd = A'C'+AB'C+A'B+BC'



Fe = A'C'+BC'



Ff = AB'+AC'+B'C'



Fg = AB'+A'B+BC'

Page 29

57

```
(S1 == 0 && S0 == 1) ? D1 :
(S1 == 1 && S0 == 0) ? D2 :
(S1 == 1 && S0 == 1) ? D3 : 1'bx;
endmodule.
```

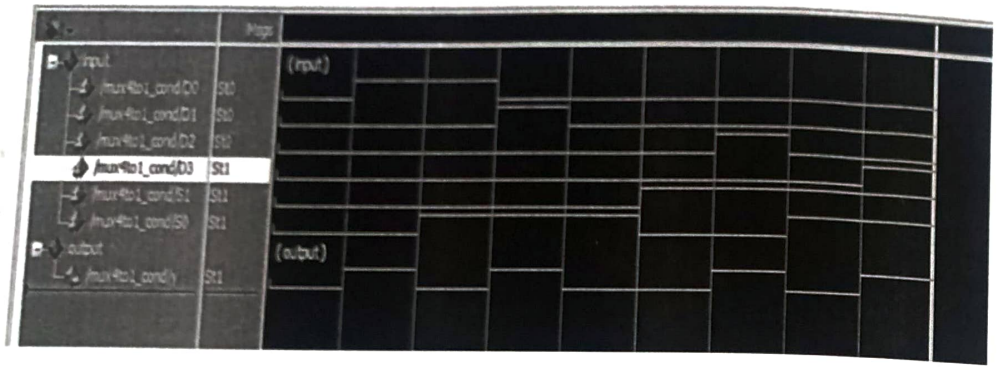
module 1:4 DEMUX in verilog using

a- Continuous assignment with logical operator.

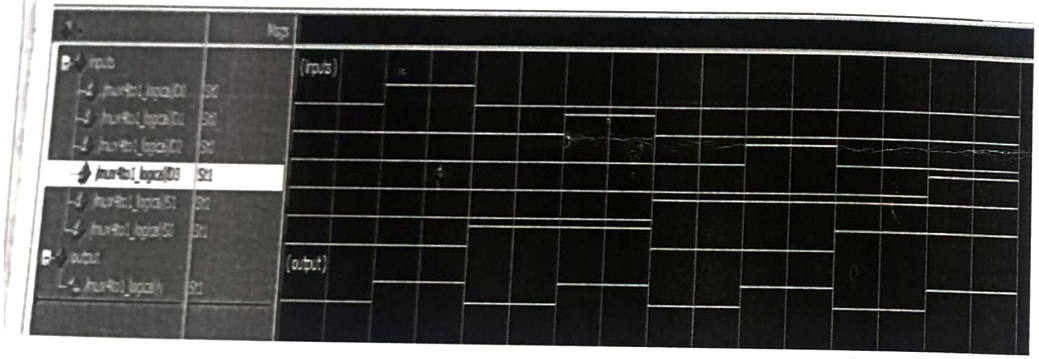
b- Continuous assignment with conditional operator.

```
'timescale 1ns/1ps
// 1:4 Demultiplexer using logical operators.
module demux 1to4 - Logical (D, S1, S0, Y0, Y1, Y2, Y3);
input D, S1, S0;
output Y0, Y1, Y2, Y3;
  assign Y0 = D & ~S1 & ~S0;
  assign Y1 = D & ~S1 & S0;
  assign Y2 = D & S1 & ~S0;
  assign Y3 = D & S1 & S0;
endmodule
// 1:4 Demultiplexer using Conditional operators.
module demux 1to4 - Cond (D, S1, S0, Y0, Y1, Y2, Y3);
  input D, S1, S0;
  output Y0, Y1, Y2, Y3;
  assign Y0 = (S1 == 0 && S0 == 0) ? D : 0;
  assign Y1 = (S1 == 0 && S0 == 1) ? D : 0;
  assign Y2 = (S1 == 1 && S0 == 0) ? D : 0;
  assign Y3 = (S1 == 1 && S0 == 1) ? D : 0;
endmodule.
```

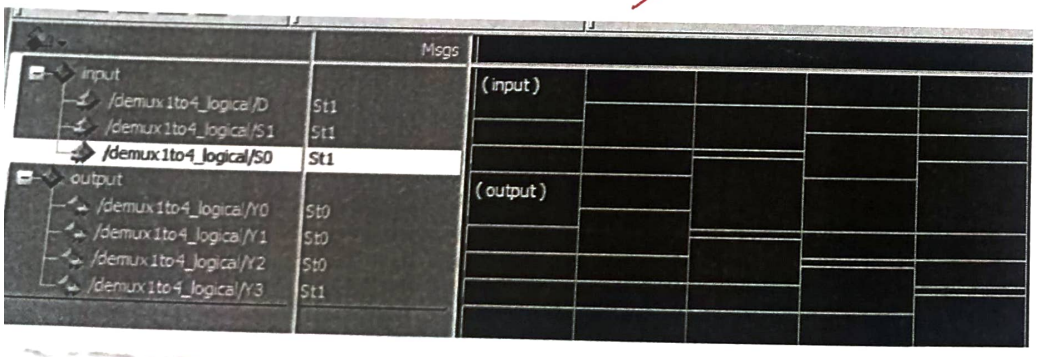
Model a 4:1 MUX in Verilog using operators, continuous assignment with conditional operators



a. 4:1 MUX using continuous assignment with logical operator



Model a 1:4 DEMUX in Verilog using a. continuous assignment with logical

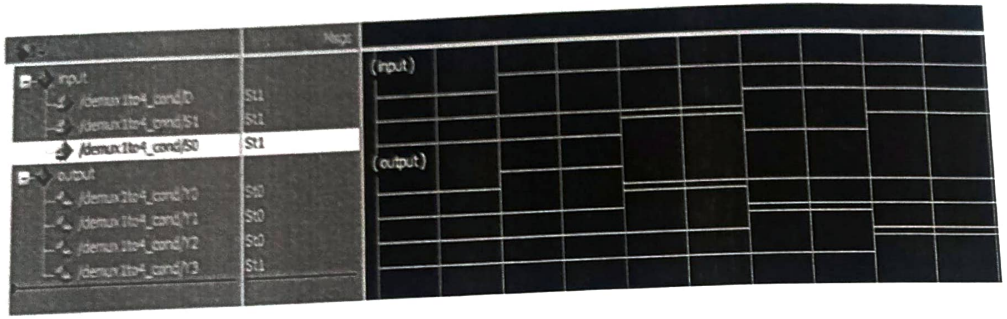


58

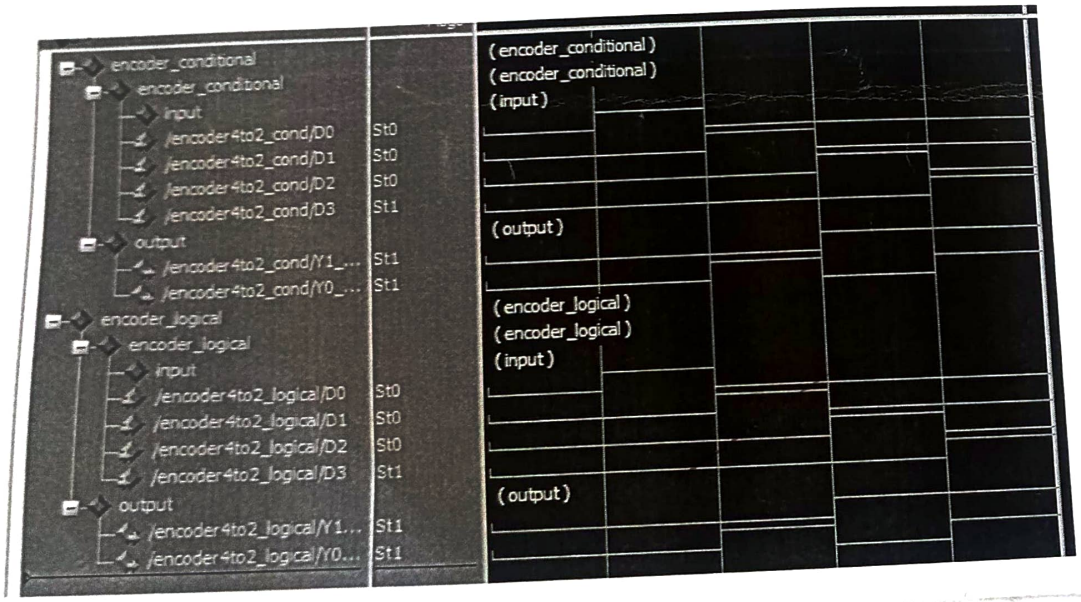
model a 4 to 2 encoder in verilog using
a. Continuous assignment with logical operator.
b. Continuous assignment with conditional operator

```
`timescale 1ns/1ps
// 4 to 2 encoder using logical operators.
module encoder 4to2 logical(Y1_log, Y0_log,
D0, D1, D2, D3);
input D0, D1, D2, D3;
output Y1_log, Y0_log;
assign Y1_log = D2|D3; // logical
assign Y0_log = D1|D3; // logical.
endmodule.
// 4 to 2 encoder using conditional operators.
module encoder 4to2_cond(Y1_cond, Y0_cond, D0,
D1, D2, D3);
input D0, D1, D2, D3;
output Y1_cond, Y0_cond;
assign Y1_cond = (D3|| D2) ? 1 : 0 // conditional operator.
assign Y0_cond = (D3|| D1) ? 1 : 0 // conditional operator.
endmodule.
model a 2 to 4 decoder in verilog using
a. Continuous assignment with logical operat
`timescale 1ns/1ps.
// 2 to 4 decoder using logical operators.
module decoder 2to4_logical(Y0_log, Y1_log, Y2_
Y3_log, A, B);
output Y0_log, Y1_log, Y2_log, Y3_log.
input A, B;
```

Model a 1:4 DEMUX in Verilog using continuous assignment with conditional operators



Model a 4 to 2 encoder in Verilog using a. continuous assignment with logical operators b. continuous assignment with conditional operators

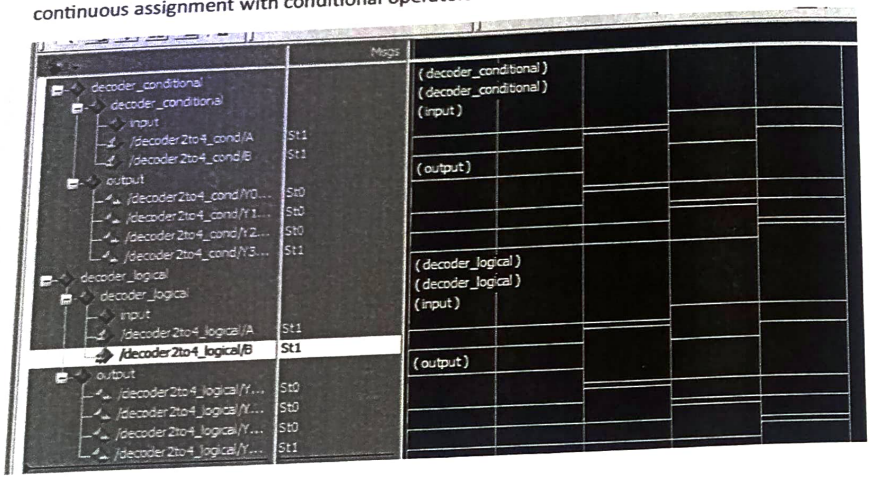



```
assign Y0_log = ~A & ~B;
assign Y1_log = ~A & B;
assign Y2_log = A & ~B;
assign Y3_log = A & B;
endmodule

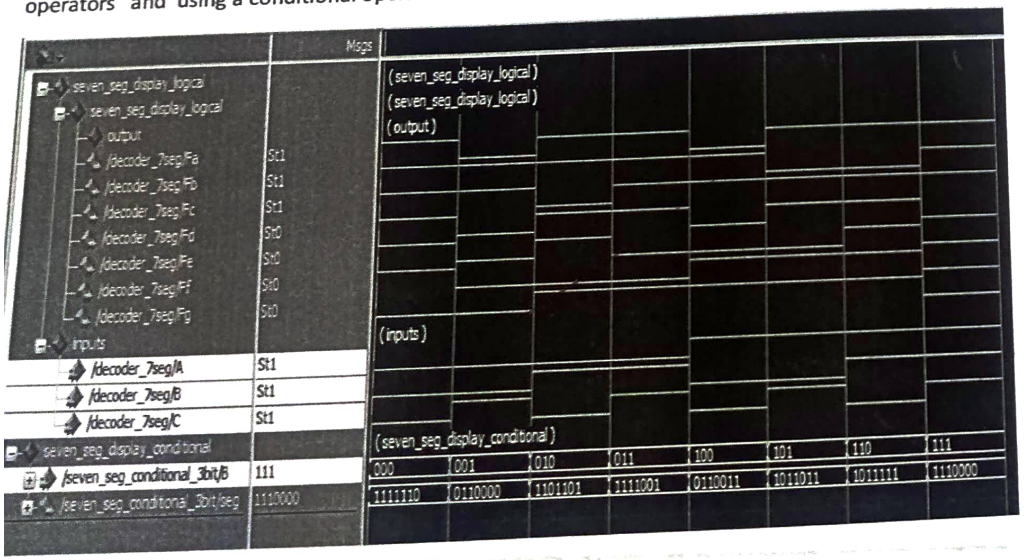
// 2to4 Decoder using Conditional operator
module decoder 2to4_Cond (Y0_Cond, Y1_Cond,
Y2_Cond, Y3_Cond, A, B);
  input A, B;
  output Y0_Cond, Y1_Cond, Y2_Cond, Y3_Cond;
  assign Y0_Cond = (A==0 && B==0)? 1:0;
  assign Y1_Cond = (A==0 && B==1)? 1:0;
  assign Y2_Cond = (A==1 && B==0)? 1:0;
  assign Y3_Cond = (A==1 && B==1)? 1:0;
endmodule.

model a 7-Segment Display Decoder.
in Verilog using:
a. Continuous assignment with logical operator
module decoder_7seg (output wire Fa, Fb, Fc, Fd,
Fe, Ff, Fg, input wire A, B, C);
  assign Fa = (~A & ~C) | (B) | (A & C);
  assign Fb = (~B & ~C) | (~A) | (B & C);
  assign Fc = (A) | (~B) | (C);
  assign Fd = (~A & ~C) | (~A & B) | (B & ~C) |
              (A & ~B & C);
  assign Fe = (~A & ~C) | (B & ~C);
  assign Ff = (~B & ~C) | (A & ~C) | (A & ~B);
  assign Fg = (~A & B) | (A & ~C) | (A & ~B);
endmodule.
```

Model a 2 to 4 decoder in Verilog using a. continuous assignment with logical operators b. continuous assignment with conditional operators



Model a 7-Segment Display Decoder in Verilog using a. continuous assignment with logical operators and using a conditional operator



b- Continuous assignment with conditional operators

```
module Seven_Seg_Conditional_3bit (input [2:0] B,
output [6:0] seg);
assign seg = (B==3'd0) ? 7'b1111110 : //0
(B==3'd1) ? 7'b0110000 : //1
(B==3'd2) ? 7'b1101101 : //2
(B==3'd3) ? 7'b1111001 : //3
(B==3'd4) ? 7'b0110011 : //4
(B==3'd5) ? 7'b1011011 : //5
(B==3'd6) ? 7'b1011111 : //6
(B==3'd7) ? 7'b1110000 : //7
7'b0000000; // Blank for invalid input
endmodule.
```

Result:

All circuits 4:1 MUX, 1:4 DEMUX, 4 to 2 encoder, Decoder and 7-Segment display Decoder were successfully modeled and simulated using conditional assignment with logical and conditional operators in Verilog HDL.